

24D.1 — crear las transformaciones

DROP TABLE IF EXISTS

bu_frd.model_behavior_features_transformed_2022_2025;

CREATE TABLE

bu_frd.model_behavior_features_transformed_2022_2025

USING PARQUET

AS

SELECT

*,

/* =====
ACTIVE ACCOUNTS
Magnitude of change
===== */

ABS(
 active_accounts_change_vs_baseline
) AS abs_active_accounts_change,

/* =====
BALANCE
Magnitude, log compressed
===== */

CASE
 WHEN balance_change_vs_baseline IS NULL
 THEN NULL

 ELSE LN(
 1.0 + ABS(balance_change_vs_baseline)
)
END AS log1p_abs_balance_change

FROM

bu_frd.model_behavior_features_selected_2022_2025;

QA estructural:

SELECT

COUNT(*) AS rows,
COUNT(DISTINCT party_id) AS parties,

SUM(

```

CASE
  WHEN abs_active_accounts_change IS NULL
  THEN 1 ELSE 0
END
) AS abs_active_missing,

SUM(
CASE
  WHEN log1p_abs_balance_change IS NULL
  THEN 1 ELSE 0
END
) AS log_balance_missing

FROM
  bu_frd.model_behavior_features_transformed_2022_2025;

```

Esperamos:

```

rows          688,589
parties        688,589
abs_active_missing    0
log_balance_missing  64,915 aproximadamente en TRAIN
                    + correspondientes VALIDATION/TEST

```

24D.2 — deciles de las dos transformaciones en TRAIN

WITH long_features AS (

```

SELECT
  party_id,
  label_3m,

  STACK(
    2,

    'abs_active_accounts_change',
    CAST(abs_active_accounts_change AS DOUBLE),

    'log1p_abs_balance_change',
    CAST(log1p_abs_balance_change AS DOUBLE)

  ) AS (feature_name, feature_value)

```

```

FROM
  bu_frd.model_behavior_features_transformed_2022_2025

```

```
WHERE model_split = 'TRAIN'
),

available AS (

SELECT *
FROM long_features

WHERE feature_value IS NOT NULL
),
```

```
ranked AS (

SELECT
    *,

    NTILE(10) OVER (
        PARTITION BY feature_name
        ORDER BY feature_value
    ) AS decile

FROM available
),
```

```
baseline AS (

SELECT
    feature_name,

    AVG(
        CAST(label_3m AS DOUBLE)
    ) AS available_positive_rate

FROM available

GROUP BY feature_name
)
```

```
SELECT
    r.feature_name,
    r.decile,

    COUNT(*) AS parties,
    SUM(r.label_3m) AS cases,

    MIN(r.feature_value) AS min_value,
```

```

percentile_approx(
    r.feature_value,
    0.50
) AS median_value,

MAX(r.feature_value) AS max_value,

ROUND(
    100.0 *
    AVG(CAST(r.label_3m AS DOUBLE)),
    4
) AS positive_rate_pct,

ROUND(
    AVG(CAST(r.label_3m AS DOUBLE))
    /
    MAX(b.available_positive_rate),
    4
) AS lift_vs_available_population

FROM ranked r

INNER JOIN baseline b
    ON r.feature_name = b.feature_name

GROUP BY
    r.feature_name,
    r.decile

ORDER BY
    r.feature_name,
    r.decile;

```